

# mjc\_sitl\_px4

Three controllers, flown as ROS 2 nodes

PD, MPC and a trained PPO policy —  
what runs, what does not, and how to run it.

## The problem

You write a drone controller in MuJoCo. It works. Now it has to run as a ROS 2 node, because that is what runs on hardware.

The normal path is to open the MuJoCo script on one monitor, an empty ROS node on the other, and retype it.

**That rewrite is where the bugs come from** — not because people are careless, but because nothing checks the result. Two pieces of code that are meant to do the same thing, and no way to tell whether they do.

### What this does

Runs your controller in simulation, runs the *same* controller through ROS 2, and compares the two flights numerically.

# The demo

```
$ cd ~/mjc_sitl_px4  
$ ./demo.sh show
```

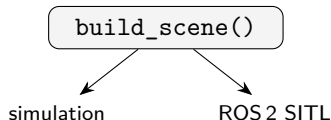
Three recorded flights, figure-8, 24 s, each through the real ROS 2 stack:

|            | tracking RMSE  | solve p95 |                                |
|------------|----------------|-----------|--------------------------------|
| <b>PD</b>  | 0.119 m        | 0.09 ms   | position → tilt → torque       |
| <b>MPC</b> | <b>0.078 m</b> | 8.3 ms    | convex QP, hover linearisation |
| <b>PPO</b> | 0.133 m        | 0.43 ms   | residual policy over PD        |

MPC tracks best and costs  $\sim 90\times$  more compute. PPO is worst here, which is expected — with no wind, PD is already near-optimal on this task and a residual policy mostly adds exploration noise.

## Why they agree: one model, two harnesses

The ROS 2 node does not have its own copy of the scene. It calls `sim_mjc`'s own `build_scene()` — same function, same compiled MuJoCo model.



So the two differ only in transport, which step the command lands on, and executor jitter — **all properties of the deployment, none of physics.**

```
$ ./demo.sh parity
parity: OK (12/12 checks)
```

Masses, inertias, gearing, thrust limits, timestep, allocation matrix — compared between the two compiled models.

## MPC needs a slower clock — and that is the point

At 50 Hz control and a 5 ms physics step, MPC **cannot keep up in wall-clock time**. The vehicle diverges within 20 steps.

```
4800 steps -> 19 steps, crashed=True  
control solve 27.68 ms p95 (budget 20 ms)
```

The *same* controller is perfectly stable in-process. So the demo runs the sim clock at `--rate-scale 0.2`: the control loop is unchanged, the wall-clock run just takes 5× longer, and it tracks at 0.078 m.

### Why this matters

That is a real deployment constraint, not a bug. It only appears once a controller is a *real node with a real deadline* — an in-process simulation cannot show it to you.

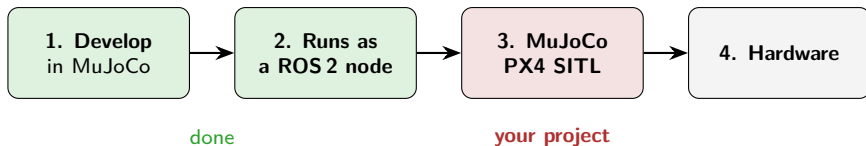
## What it *cannot* do

**This is not flight-ready PX4 code, and PX4 is not running.** Only the message *types* are borrowed. The controller publishes four thrusts in **newtons** on a custom topic.

| missing                        | consequence                                                                                                                                        |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| NED $\leftrightarrow$ ENU      | this stack is z- <b>up</b> and hovers at position[2] = +1.0; PX4 is z- <b>down</b> . The altitude loop is sign-inverted — it flies into the ground |
| OffboardControlMode            | PX4 drops offboard mode without it                                                                                                                 |
| VehicleCommand                 | nothing arms the vehicle                                                                                                                           |
| N $\rightarrow$ ActuatorMotors | PX4 wants [0, 1], calibrated per airframe                                                                                                          |
| takeoff / land / failsafe      | no mode-change handling                                                                                                                            |

Nothing here models sensor noise, state estimation (odometry is ground truth, not an EKF), actuator lag or wind. **0.19 mm means the translation is faithful** — it says nothing about robustness.

## Where it goes — and where you come in



### Why a MuJoCo PX4 SITL rather than Gazebo PX4 SITL

Gazebo would give you the PX4 protocol but a *different physics engine* — and once the engine differs you can no longer separate a deployment bug from an engine difference. The comparison you just watched stops working, right before hardware. **Stage 3 keeps both:** real PX4 protocol *and* the shared model.

## Stage 3: the work, in order

1. NED  $\leftrightarrow$  ENU and FRD  $\leftrightarrow$  FLU conversion, with a test asserting a round-trip is the identity
2. OffboardControlMode heartbeat, VehicleCommand arm / mode sequencing
3. newtons  $\rightarrow$  normalised ActuatorMotors, calibrated against the airframe
4. run it against real PX4 SITL + Gazebo
5. takeoff, land, failsafe

### Build it against the MuJoCo SITL first

Give the sim node an optional “PX4 dialect” and you can test your bridge in *one second* per attempt. Gazebo then becomes a confirmation step rather than the place you debug sign flips.

## It is not as bad as it sounds

The toolchain is already installed on this machine:

---

|                |                                                           |
|----------------|-----------------------------------------------------------|
| PX4-Autopilot  | source present, <b>already built</b>                      |
| Gazebo         | 8.9.0 on PATH                                             |
| MicroXRCEAgent | installed (the PX4 ↔ ROS 2 bridge)                        |
| px4-offboard   | a <b>working reference</b> — heartbeat, arming, setpoints |

---

There is no clever algorithm in any of this. It is careful bookkeeping against a documented protocol.

### What makes it genuinely hard

The failures are *silent*. A sign flip does not raise an exception — it crashes the vehicle. There is no stack trace to read, which is exactly why it wants tests rather than inspection.

## Running it yourself — setup

```
# 1. you need your own PX4 workspace with px4_msgs built
export PX4_WS=/path/to/your/px4_ws

# 2. sanity check -- nothing to build, the nodes run as plain
#    Python processes once ROS is sourced
cd ~/mjc_sitl_px4
python3 -m mjc_sitl parity          # expect 12/12
python3 -m pytest mjc_sitl/tests -q # expect 22 passed
```

Requirements: Python 3.10, ROS 2 Humble, mujoco>=3.3, numpy, scipy, px4\_msgs.

Optional: matplotlib + mediapy for figures and video.

On a headless machine, prefix rendering commands with MUJOCO\_GL=egl.

## Running it yourself — the three commands

```
./demo.sh parity # do sim and SITL share physics?    (5 s)
./demo.sh fly    # figure-8 in sim AND ROS 2, compare (~90 s)
./demo.sh show   # open the pre-generated artifacts (instant)
```

Or drive it directly:

```
python3 -m mjc_sitl compare --controller pd --task figure8 \
    --duration 24 --tier full --outdir demo/figure8
```

-controller takes pd, mpc, ppo or mine; -task takes hover or figure8. -tier fast skips ROS entirely and runs in about a second, which is what you want while iterating.

Stuck on something that looks like physics? Run parity first. If it says 12/12, the airframe is not your problem.

## Where to look

|                                     |                                                |
|-------------------------------------|------------------------------------------------|
| <code>README.md</code>              | what it does, what it cannot do, where it goes |
| <code>sim_mjc/CLAUDE_SIM.MD</code>  | how the simulator works                        |
| <code>mjc_sitl/scenegen.py</code>   | the shared scene — the whole idea, 150 lines   |
| <code>mjc_sitl/bridge.py</code>     | frames, units, rotor order                     |
| <code>mjc_sitl/offboard_lib/</code> | the two ROS 2 nodes                            |

The airframe is defined *once*, in `sim_mjc/sim_mjc/config.py:DRONE`. Change it there and run `python3 -m mjc_sitl sync` — never edit the generated copies by hand.

## Three things

1. Sim and SITL run the **same compiled model**.  
So a difference between them is a deployment bug, never physics.
2. It is **not** PX4-ready. The frame flip alone is fatal.  
That gap is stage 3 — and it is the project.
3. Build stage 3 **against MuJoCo first**.  
One-second feedback beats debugging sign flips in Gazebo.

Start with `README.md`, then `./demo.sh show`.